SKILL WEEK-10

SESSION-19

Task-1: To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test 4 Signal Events.

Source code:

```
  GNU nano 8.7.1                  signal_handlers.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

volatile sig_atomic_t signal_count = 0;

/* Signal handler */
void signal_handler(int sig, siginfo_t *info, void *context)
{
    signal_count++;

    printf("\n--------------------------------\n");
    printf("Signal received: %d\n", sig);

    switch (sig)
    {
        case SIGINT:
            printf("Event: Interrupt (Ctrl+C)\n");
            break;

        case SIGTERM:
            printf("Event: Termination request\n");
            break;

        case SIGUSR1:
            printf("Event: User-defined signal 1\n");
            break;

        case SIGUSR2:
            printf("Event: User-defined signal 2\n");
            break;

        default:
            printf("Event: Unknown signal\n");
    }

    if (info != NULL)
        printf("Sender PID: %d\n", info->si_pid);
```

```
  GNU nano 8.7.1                  signal_handlers.c *
int main()
{
    struct sigaction action;

    printf("Signal Handler Demonstration\n");
    printf("----------------------------\n");
    printf("Process PID: %d\n", getpid());

    /* Clear the structure */
    memset(&action, 0, sizeof(action));

    /* Configure handler */
    action.sa_sigaction = signal_handler;
    action.sa_flags = SA_SIGINFO;

    /* Initialize signal mask */
    sigemptyset(&action.sa_mask);

    /* Register four signal handlers */
    sigaction(SIGINT, &action, NULL);
    sigaction(SIGTERM, &action, NULL);
    sigaction(SIGUSR1, &action, NULL);
    sigaction(SIGUSR2, &action, NULL);

    printf("\nFour signal handlers registered successfully.\n");

    printf("\nTest the signals using another terminal:\n");
    printf("kill -USR1 %d\n", getpid());
    printf("kill -USR2 %d\n", getpid());
    printf("kill -TERM %d\n", getpid());
    printf("Or press Ctrl+C for SIGINT.\n");

    printf("\nWaiting for signals...\n");

    while (1)
    {
        pause();
    }

    return 0;
}
```

Output:

```
sathwika@DESKTOP-AA8K5MG :~$ nano signal_handlers.c
sathwika@DESKTOP-AA8K5MG :~$ gcc signal_handlers.c -o signal_handlers
sathwika@DESKTOP-AA8K5MG :~$ ./signal_handlers
Signal Handler Demonstration
----------------------------
Process PID: 504

Four signal handlers registered successfully.

Test the signals using another terminal:
kill -USR1 504
kill -USR2 504
kill -TERM 504
Or press Ctrl+C for SIGINT.

Waiting for signals...
```

Observation:

The program successfully registers and handles **four signal events** using sigaction(). Signal information is received through siginfo_t, interrupts are handled without crashing the program, and the program remains stable while waiting for additional signals.

Task-2: To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt 4 Handling.

Source code:

```
  GNU nano 8.7.1                  sigint_shell.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>

volatile sig_atomic_t foreground_pid = -1;

/* SIGINT handler for the shell */
void handle_sigint(int sig)
{
    printf("\nShell: SIGINT received.\n");

    if (foreground_pid > 0)
    {
        printf("Shell: Forwarding SIGINT to foreground job PID %d\n",
               foreground_pid);

        kill(foreground_pid, SIGINT);
    }
    else
    {
        printf("Shell: No foreground job. Shell remains active.\n");
    }

    fflush(stdout);
}

int main()
{
    struct sigaction action;
    pid_t pid;
    int status;

    printf("Simple SIGINT Shell\n");
    printf("-------------------\n");
    printf("Shell PID: %d\n", getpid());

    /* Configure SIGINT handler */
    memset(&action, 0, sizeof(action));
    action.sa_handler = handle_sigint;
```

```
  GNU nano 8.7.1                  sigint_shell.c *
        printf("Foreground Job: PID = %d\n", getpid());
        printf("Foreground Job: Running...\n");

        for (int i = 1; i <= 20; i++)
        {
            printf("Foreground Job: Working... %d/20\n", i);
            sleep(1);
        }

        printf("Foreground Job: Completed normally.\n");
        exit(0);
    }

    /* Parent / shell */
    foreground_pid = pid;

    printf("Shell: Foreground job PID = %d\n", pid);
    printf("Shell: Press Ctrl+C to interrupt the foreground job.\n");

    /* Wait for foreground job */
    waitpid(pid, &status, 0);

    /* Job has finished */
    foreground_pid = -1;

    if (WIFSIGNALED(status))
    {
        printf("\nJob State: Terminated by signal %d\n",
               WTERMSIG(status));
    }
    else if (WIFEXITED(status))
    {
        printf("\nJob State: Completed\n");
        printf("Exit Status: %d\n", WEXITSTATUS(status));
    }

    printf("\nShell is still running.\n");
    printf("SIGINT handling test completed.\n");

    return 0;
}
```

Output:

```
sathwika@DESKTOP-AA8K5MG :~$ nano sigint_shell.c
sathwika@DESKTOP-AA8K5MG :~$ gcc sigint_shell.c -o sigint_shell
sathwika@DESKTOP-AA8K5MG :~$ ./sigint_shell
Simple SIGINT Shell
-------------------
Shell PID: 667

Starting foreground job...
Shell: Foreground job PID = 668
Shell: Press Ctrl+C to interrupt the foreground job.
Foreground Job: PID = 668
Foreground Job: Running...
Foreground Job: Working... 1/20
Foreground Job: Working... 2/20
Foreground Job: Working... 3/20
Foreground Job: Working... 4/20
Foreground Job: Working... 5/20
Foreground Job: Working... 6/20
Foreground Job: Working... 7/20
Foreground Job: Working... 8/20
Foreground Job: Working... 9/20
Foreground Job: Working... 10/20
Foreground Job: Working... 11/20
Foreground Job: Working... 12/20
Foreground Job: Working... 13/20
Foreground Job: Working... 14/20
Foreground Job: Working... 15/20
Foreground Job: Working... 16/20
Foreground Job: Working... 17/20
Foreground Job: Working... 18/20
Foreground Job: Working... 19/20
Foreground Job: Working... 20/20
Foreground Job: Completed normally.

Job State: Completed
Exit Status: 0

Shell is still running.
SIGINT handling test completed.
sathwika@DESKTOP-AA8K5MG :~$
```

Observation:

When Ctrl+C is pressed, SIGINT is captured by the shell and forwarded to the foreground process. The foreground job terminates, while the shell itself remains protected. The shell then checks the child status and updates the job state.

SESSION-20

Task-1: To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

Source code:

```
  GNU nano 8.7.1                  sigtstp_jobs.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>

typedef struct {
    int job_id;
    pid_t pid;
    char command[50];
    char state[20];
} Job;

Job job;
volatile sig_atomic_t foreground_pid = -1;

/* Handle Ctrl+Z */
void handle_sigtstp(int sig)
{
    printf("\nShell: SIGTSTP received.\n");

    if (foreground_pid > 0) {
        printf("Shell: Suspending foreground job PID %d\n",
               foreground_pid);

        kill(foreground_pid, SIGTSTP);
    }
}

void display_job()
{
    printf("\n------ Job Table ------\n");
    printf("Job ID\tPID\tState\t\tCommand\n");
    printf("-----------------------------------------\n");

    printf("[%d]\t%d\t%-12s\t%s\n",
           job.job_id,
           job.pid,
           job.state,
           job.command);
```

```
      display_job();

      printf("\nJob state preserved.\n");
      printf("Press ENTER to resume the job...\n");

      getchar();

      printf("\nSending SIGCONT to Job [%d]...\n", job.job_id);

      if (kill(pid, SIGCONT) == 0) {
          strcpy(job.state, "Running");
          printf("Job [%d] resumed successfully.\n", job.job_id);
      }

      display_job();

      /* Wait for completion */
      waitpid(pid, &status, 0);

      if (WIFEXITED(status)) {
          strcpy(job.state, "Completed");
      }
      else if (WIFSIGNALED(status)) {
          strcpy(job.state, "Terminated");
      }

      display_job();
  }

  else if (WIFEXITED(status)) {

      strcpy(job.state, "Completed");

      display_job();
  }

  printf("\nSuspension workflow completed.\n");

  return 0;
```

Output:

```
sathwika@DESKTOP-AA8K5MG :~$ nano sigtstp_jobs.c
sathwika@DESKTOP-AA8K5MG :~$ gcc sigtstp_jobs.c -o sigtstp_jobs
sathwika@DESKTOP-AA8K5MG :~$ ./sigtstp_jobs
SIGTSTP Job Suspension Demo
---------------------------
Shell PID: 795

Job [1] created with PID 796
Press Ctrl+Z to suspend the job.

------ Job Table ------
Job ID  PID     State           Command
---------------------------------------
[1]     796     Running         foreground-task
---------------------------------------
Foreground Job: PID = 796
Foreground Job: Running...
Foreground Job: Working 1/15
Foreground Job: Working 2/15
Foreground Job: Working 3/15
Foreground Job: Working 4/15
Foreground Job: Working 5/15
Foreground Job: Working 6/15
Foreground Job: Working 7/15
Foreground Job: Working 8/15
Foreground Job: Working 9/15
Foreground Job: Working 10/15
Foreground Job: Working 11/15
Foreground Job: Working 12/15
Foreground Job: Working 13/15
Foreground Job: Working 14/15
Foreground Job: Working 15/15
Foreground Job: Completed.

------ Job Table ------
Job ID  PID     State           Command
---------------------------------------
[1]     796     Completed       foreground-task
---------------------------------------

Suspension workflow completed.
sathwika@DESKTOP-AA8K5MG :~$
```

Observation:

When Ctrl+Z is pressed, SIGTSTP is captured by the shell and forwarded to the foreground process. The process changes from **Running** to **Stopped**, and the job table is updated. When SIGCONT is sent, the process resumes and continues its previous execution before eventually reaching the **Completed** state.

Task-2: To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

Source code:

```
  GNU nano 8.7.1                  process_groups.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>
#include <errno.h>

int main()
{
    pid_t shell_pgid;
    pid_t child1, child2;
    pid_t pgid;
    int status;

    shell_pgid = getpgrp();

    printf("Process Group and Terminal Control Demo\n");
    printf("---------------------------------------\n");
    printf("Shell PID       : %d\n", getpid());
    printf("Shell PGID      : %d\n", shell_pgid);
    printf("Terminal PGID   : %d\n", tcgetpgrp(STDIN_FILENO));

    /*
     * Create first child
     */
    child1 = fork();

    if (child1 < 0) {
        perror("fork");
        return 1;
    }

    if (child1 == 0) {
        /*
         * First child creates a new process group.
         * Its PID becomes the PGID.
         */
        setpgid(0, 0);

        printf("\nChild 1:\n");
        printf("PID  = %d\n", getpid());
```

```
    printf("Terminal foreground PGID: %d\n",
           tcgetpgrp(STDIN_FILENO));

    /*
     * Send SIGCONT to the complete process group.
     */
    printf("\nSending SIGCONT to process group...\n");

    if (kill(-pgid, SIGCONT) == -1) {
        perror("kill");
    } else {
        printf("SIGCONT sent to process group %d\n", pgid);
    }

    /*
     * Wait for both processes.
     */
    waitpid(child1, &status, 0);
    waitpid(child2, &status, 0);

    /*
     * Restore terminal ownership to the shell.
     */
    printf("\nRestoring terminal ownership...\n");

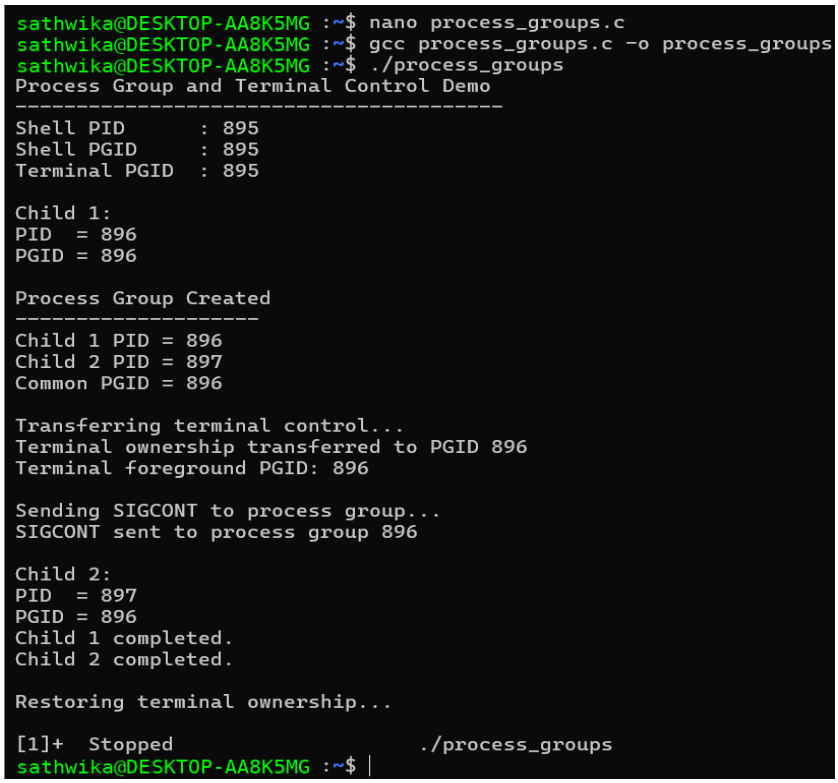
    if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1) {
        perror("tcsetpgrp");
    } else {
        printf("Terminal returned to shell PGID %d\n",
               shell_pgid);
    }

    printf("Current terminal PGID: %d\n",
           tcgetpgrp(STDIN_FILENO));

    printf("\nProcess group test completed successfully.\n");

    return 0;
}
```

Output:



Observation:

Two child processes are created and placed into the same process group. The terminal is transferred to that group using tcsetpgrp(). A signal can then be delivered to the complete group using a negative PGID. After the processes finish, terminal ownership is restored to the shell.